

```
"""
```

```
=====
BTC Trading Bot - Coinbase Advanced API
Strategy: 9 EMA / 20 EMA Crossover + RSI
Author: Built for you by Claude
=====
```

SETUP INSTRUCTIONS (do these FIRST):

1. Go to Coinbase Advanced → Settings → API
2. Create API key with "Trade" permission
3. Copy your API Key and API Secret below
4. Install requirements: `pip install requests pandas pandas-ta coinbase-advanced-`

 WARNING: This bot uses real money. Start with \$25 and monitor it closely.

```
"""
```

```
import os
import time
import hmac
import hashlib
import json
import requests
import pandas as pd
import pandas_ta as ta
from datetime import datetime, timezone

# -----
#  YOUR CREDENTIALS - FILL THESE IN
# -----
API_KEY = os.environ.get("CB_API_KEY", "")
API_SECRET = os.environ.get("CB_API_SECRET", "")
```

```
if not API_KEY or not API_SECRET:
    raise ValueError(" CB_API_KEY and CB_API_SECRET must be set in Railway envi
```

```
# -----
#  BOT SETTINGS - ADJUST AS YOU LIKE
# -----
SYMBOL = "BTC-USD" # Trading pair
CANDLE_INTERVAL = "FIVE_MINUTE" # 5-min candles (best for scalping)
TRADE_AMOUNT = 20.0 # USD per trade (keep at $20 to start)
TAKE_PROFIT_PCT = 0.04 # Take profit at +4%
STOP_LOSS_PCT = 0.02 # Stop loss at -2%
EMA_FAST = 9 # Fast EMA period
```

```

EMA_SLOW          = 20          # Slow EMA period
RSI_PERIOD        = 14          # RSI period
RSI_BULL          = 50          # RSI must be above this to buy
LOOP_SECONDS      = 60          # Check every 60 seconds

# -----
# DO NOT EDIT BELOW THIS LINE
# -----

BASE_URL = "https://api.coinbase.com"

class CoinbaseBot:
    def __init__(self):
        self.in_trade      = False
        self.entry_price    = None
        self.position_size  = None # BTC amount held
        self.take_profit    = None
        self.stop_loss      = None
        self.trade_count    = 0
        self.wins            = 0
        self.losses         = 0
        self.pnl_total      = 0.0

    # — Auth headers for Coinbase Advanced —
    def _headers(self, method, path, body=""):
        timestamp = str(int(time.time()))
        message   = timestamp + method + path + body
        signature  = hmac.new(
            self.API_SECRET.encode("utf-8"),
            message.encode("utf-8"),
            digestmod=hashlib.sha256
        ).hexdigest()
        return {
            "CB-ACCESS-KEY":      API_KEY,
            "CB-ACCESS-SIGN":     signature,
            "CB-ACCESS-TIMESTAMP": timestamp,
            "Content-Type":       "application/json",
        }

    # — Fetch OHLCV candles —
    def get_candles(self):
        path = f"/api/v3/brokerage/products/{SYMBOL}/candles"
        params = {"granularity": CANDLE_INTERVAL, "limit": 100}
        headers = self._headers("GET", path)
        resp = requests.get(BASE_URL + path, headers=headers, params=params)
        resp.raise_for_status()
        data = resp.json().get("candles", [])

```

```

df = pd.DataFrame(data, columns=["start", "low", "high", "open", "close", "vol
df = df.astype({"open": float, "high": float, "low": float,
                "close": float, "volume": float})
df = df.sort_values("start").reset_index(drop=True)
return df

# — Calculate indicators —
def get_signals(self, df):
    df["ema_fast"] = ta.ema(df["close"], length=EMA_FAST)
    df["ema_slow"] = ta.ema(df["close"], length=EMA_SLOW)
    df["rsi"]      = ta.rsi(df["close"], length=RSI_PERIOD)
    latest = df.iloc[-1]
    prev   = df.iloc[-2]

    ema_cross_up   = (prev["ema_fast"] <= prev["ema_slow"]) and \
                    (latest["ema_fast"] > latest["ema_slow"])
    ema_cross_down = (prev["ema_fast"] >= prev["ema_slow"]) and \
                    (latest["ema_fast"] < latest["ema_slow"])
    rsi_bull       = latest["rsi"] > RSI_BULL
    price          = latest["close"]

    return {
        "price":          price,
        "ema_fast":       latest["ema_fast"],
        "ema_slow":       latest["ema_slow"],
        "rsi":            latest["rsi"],
        "buy_signal":     ema_cross_up and rsi_bull,
        "sell_signal":    ema_cross_down,
    }

# — Get current BTC price —
def get_price(self):
    path = f"/api/v3/brokerage/products/{SYMBOL}"
    headers = self._headers("GET", path)
    resp = requests.get(BASE_URL + path, headers=headers)
    resp.raise_for_status()
    return float(resp.json()["price"])

# — Place market buy order —
def buy(self, usd_amount):
    path = "/api/v3/brokerage/orders"
    body = json.dumps({
        "client_order_id": f"bot-buy-{int(time.time())}",
        "product_id":      SYMBOL,
        "side":            "BUY",
        "order_configuration": {
            "market_market_ioc": {

```

```

        "quote_size": str(round(usd_amount, 2))
    }
}

))
headers = self._headers("POST", path, body)
resp = requests.post(BASE_URL + path, headers=headers, data=body)
resp.raise_for_status()
return resp.json()

# — Place market sell order —
def sell(self, btc_amount):
    path = "/api/v3/brokerage/orders"
    body = json.dumps({
        "client_order_id": f"bot-sell-{int(time.time())}",
        "product_id": SYMBOL,
        "side": "SELL",
        "order_configuration": {
            "market_market_ioc": {
                "base_size": str(round(btc_amount, 8))
            }
        }
    })
    headers = self._headers("POST", path, body)
    resp = requests.post(BASE_URL + path, headers=headers, data=body)
    resp.raise_for_status()
    return resp.json()

# — Logging —
def log(self, msg):
    ts = datetime.now(timezone.utc).strftime("%H:%M:%S UTC")
    print(f"[{ts}] {msg}")

# — Print status —
def print_status(self, sig):
    self.log(
        f"BTC=${sig['price']:.2f} | "
        f"EMA9={sig['ema_fast']:.1f} EMA20={sig['ema_slow']:.1f} | "
        f"RSI={sig['rsi']:.1f} | "
        f"In Trade: {'YES' if self.in_trade else 'NO'} | "
        f"P&L: ${self.pnl_total:+.2f}"
    )

# — Main loop —
def run(self):
    self.log("🤖 BTC Bot started — Coinbase Advanced")
    self.log(f"Strategy: {EMA_FAST}/{EMA_SLOW} EMA Crossover + RSI>{RSI_BULL}")
    self.log(f"Trade size: ${TRADE_AMOUNT} | TP: +{TAKE_PROFIT_PCT*100}% | SL

```

```

self.log("-" * 60)

while True:
    try:
        df = self.get_candles()
        sig = self.get_signals(df)
        self.print_status(sig)

        # — If in a trade, check TP/SL —
        if self.in_trade:
            price = sig["price"]
            if price >= self.take_profit:
                pnl = (price - self.entry_price) * self.position_size
                self.log(f"✅ TAKE PROFIT hit at ${price:,.2f} | PnL: +${pnl}")
                self.sell(self.position_size)
                self.pnl_total += pnl
                self.wins += 1
                self.in_trade = False

            elif price <= self.stop_loss:
                pnl = (price - self.entry_price) * self.position_size
                self.log(f"🛑 STOP LOSS hit at ${price:,.2f} | PnL: ${pnl}")
                self.sell(self.position_size)
                self.pnl_total += pnl
                self.losses += 1
                self.in_trade = False

            elif sig["sell_signal"]:
                pnl = (price - self.entry_price) * self.position_size
                self.log(f"📉 SELL SIGNAL — exiting at ${price:,.2f} | PnL: ${pnl}")
                self.sell(self.position_size)
                self.pnl_total += pnl
                if pnl > 0: self.wins += 1
                else: self.losses += 1
                self.in_trade = False

        # — Not in trade, check for buy —
        else:
            if sig["buy_signal"]:
                price = sig["price"]
                btc_qty = TRADE_AMOUNT / price
                self.log(f"📈 BUY SIGNAL — entering at ${price:,.2f}")
                self.buy(TRADE_AMOUNT)
                self.entry_price = price
                self.position_size = btc_qty
                self.take_profit = price * (1 + TAKE_PROFIT_PCT)
                self.stop_loss = price * (1 - STOP_LOSS_PCT)

```

```

        self.in_trade      = True
        self.trade_count   += 1
        self.log(f"    TP: ${self.take_profit:,.2f} | SL: ${self.s

    self.log(f"    Trades: {self.trade_count} | W: {self.wins} L: {sel

except KeyboardInterrupt:
    self.log("Bot stopped by user.")
    self.log(f"Final P&L: ${self.pnl_total:+.2f} | W:{self.wins} L:{s
    break

except Exception as e:
    self.log(f"⚠️ Error: {e} - retrying in 30s")
    time.sleep(30)
    continue

time.sleep(LOOP_SECONDS)

if __name__ == "__main__":
    bot = CoinbaseBot()
    bot.run()

```